

Panduan dosen Lab 05 - Dockerfile dan Cloud

Notes API

Durasi demo: 90 menit. **Capaian:** mahasiswa dapat menerangkan Dockerfile multi-stage, image versus container, build/run time, port mapping, user non-root, validasi API, dan mengapa data in-memory hilang saat container diganti. Repo template mandiri: [SeedFlora/meet5CloudService](#). Gunakan [modul mahasiswa](#) sebagai petunjuk yang menyertakan kunci tantangan `/stats`.

Pra-kelas dan batas lingkungan

1. Pastikan Docker Engine/Desktop hidup: `docker version`. Port host `127.0.0.1:8000` perlu kosong. Lab 06 memakai port sama; hentikan hanya container Lab 05 milik praktik ini (`docker rm -f cloudlab-api`) sebelum Lab 06 dimulai.
2. Dari **root repo Lab 05** jalankan `docker build -t cloud-notes-api:lab05 ..`. Dockerfile mengunduh `python:3.12-slim` dan memasang dependency yang dipin dalam `requirements.txt`; siapkan jaringan pada build pertama.
3. Siapkan dua terminal: terminal Docker/log dan terminal HTTP. PowerShell menggunakan `Invoke-RestMethod` atau `curl.exe`; Bash/Git Bash memakai `curl`.
4. Kode starter belum berisi endpoint `/stats`. Kunci ada di modul, lalu telah dibuild dan diuji pada salinan sementara agar mahasiswa tetap dapat mengerjakannya sendiri.

Alur 90 menit

Menit	Dosen	Mahasiswa	Bukti/checkpoint
0-10	Hubungkan Lab 04 image/container dengan Dockerfile.	Bedakan instruksi build dan run.	Tabel RUN/COPY/CMD/ENV.
10-23	Tunjukkan Dockerfile builder dan runtime, <code>.dockerignore</code> .	Prediksi isi image akhir.	Dependency dan kode, tanpa file <code>.env</code> .
23-35	Build image dan inspect ukuran/user.	Jalankan build bertag.	Image <code>cloud-notes-api:lab05</code> .
35-48	Run container, <code>docker ps/logs/exec id</code> .	Buka <code>/docs</code> .	UID 10001, port 8000, OpenAPI.
48-63	Demo GET health/runtime, POST/GET notes, 422.	Ulangi HTTP dan catat status.	200/201/200/422; teks spasi ditolak.
63-72	Hapus dan buat ulang container.	Cek daftar kembali kosong.	<code>GET /notes = []</code> .
72-84	Tantangan <code>/stats</code> dan kunci, build tag baru.	Implementasikan/test dua catatan.	Awal 0/0, akhir 2/11.
84-90	Review lima jawaban, Git, cleanup.	Commit kode/laporan pribadi.	Bukti tidak memuat secret.

Demo command yang dapat disalin

Dari root repo Lab 05:

```
docker version
docker build -t cloud-notes-api:lab05 .
docker image ls cloud-notes-api
docker run --name clouddlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:lab05
docker ps --filter name=clouddlab-api
docker logs clouddlab-api
docker exec clouddlab-api id
docker image inspect cloud-notes-api:lab05 --format '{{.Size}} bytes; user={{.Config.User}}'
```

`docker build` menjalankan tahap builder (pip ke `/opt/venv`) dan runtime (salin `venv`+kodenya). `docker run` membuat container; `-d` melepaskan terminal, `-p` memetakan port host ke container, `-e` menyuntik `APP_ENV`. `docker exec id` memperlihatkan UID 10001, bukan root. `docker logs` memperlihatkan proses Uvicorn. Di browser buka `http://127.0.0.1:8000/docs`.

PowerShell untuk demo API:

```
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-RestMethod http://127.0.0.1:8000/runtime
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
\'{"title":"Docker","content":"Catatan kelas"}\'
Invoke-RestMethod http://127.0.0.1:8000/notes
try { Invoke-WebRequest http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body
\'{"title":" ","content":"uji"}\' -UseBasicParsing } catch { [int]$_Exception.Response.StatusCode }
```

GET `health/runtime` harus HTTP 200; POST valid HTTP 201; GET `notes` memuat catatan yang baru; judul spasi saja HTTP 422. Validator `reject_blank_text` memangkas spasi dan menolak hasil kosong. Tes GET `/notes/999999` harus HTTP 404. Untuk membuktikan in-memory, jalankan `docker rm -f clouddlab-api`, ulangi `docker run` yang sama, tunggu API siap, lalu GET `/notes`: hasil `[]`. Ini membuat **container baru**, bukan sekadar restart container lama.

Kunci lengkap tantangan /stats

Tambahkan ke akhir `app/main.py` pada salinan mahasiswa:

```
@app.get("/stats")
def stats() -> dict[str, int]:
    with _lock:
        titles = [item["title"] for item in _notes.values()]
        return {
            "note_count": len(titles),
            "longest_title_chars": max((len(title) for title in titles), default=0),
        }
```

`_lock` adalah lock yang sama dengan POST/GET agar snapshot konsisten. `default=0` mencegah `ValueError` ketika belum ada catatan. Build dengan `docker build -t cloud-notes-api:stats .`, hapus container lama, jalankan `cloud-notes-api:stats`, dan periksa:

```
curl -fsS http://127.0.0.1:8000/stats
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d
\'{"title":"Ops","content":"Pertama"}\'
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d
\'{"title":"Incident 42","content":"Kedua"}\'
curl -fsS http://127.0.0.1:8000/stats
```

Hasil tepat **0/0** sebelum POST dan **2/11** setelahnya. Kunci ini diuji pada image solusi yang dibangun dari salinan kode: dua POST HTTP 201, stats HTTP 200, dan judul spasi HTTP 422. Jika `/stats` masih 404, periksa image pada `docker ps --format '{{.Image}}'`: mahasiswa sering mengedit kode host tetapi tetap menjalankan image lama.

Screenshot, command, fungsi, dan cara membaca

```
Cloud Services - hasil perintah lokal

Image build dan user runtime
Output aktual dari sesi uji, ditata ulang agar mudah dibaca pada modul.

docker image inspect; docker exec id; docker ps

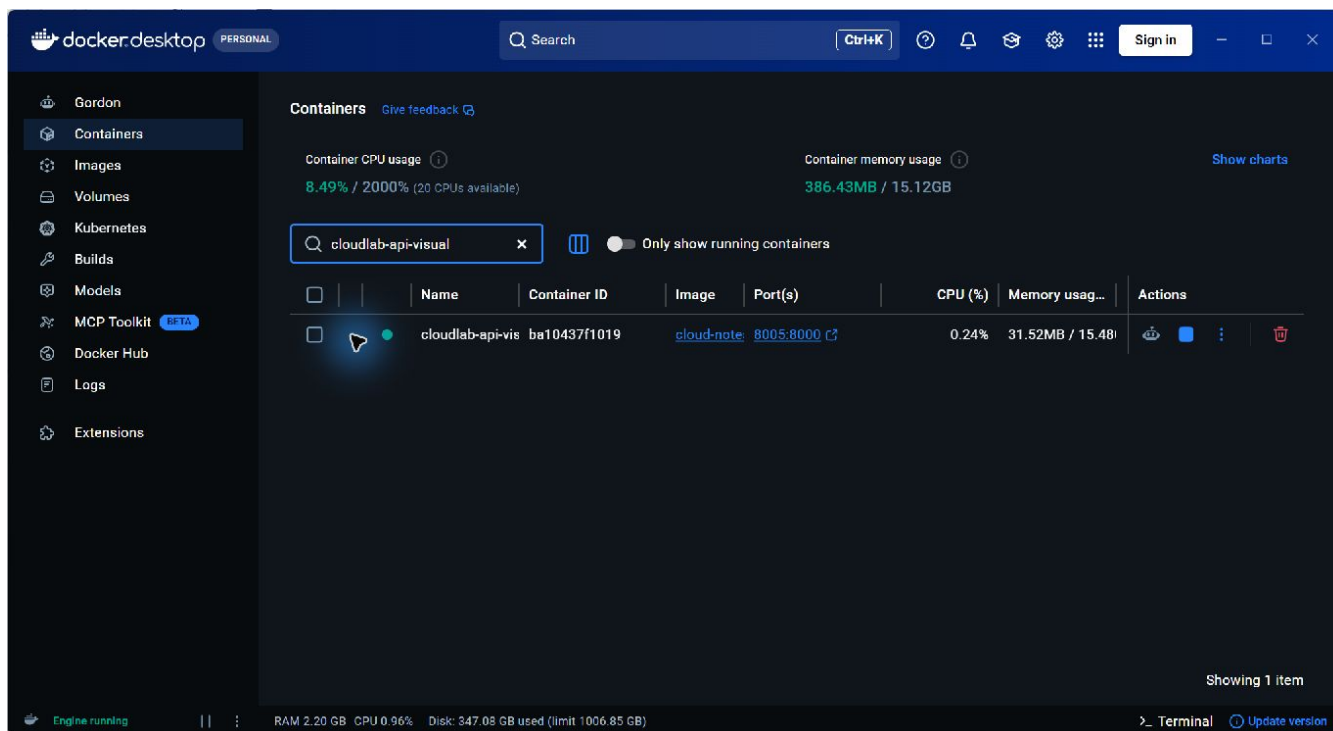
cloud-notes-api:lab05 | user=appuser | 52164335 bytes

docker exec cloudlab-api id
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)

NAMES          IMAGE          PORTS
cloudlab-api   cloud-notes-api:lab05   127.0.0.1:8000->8000/tcp
```

Image dan user container pada uji awal

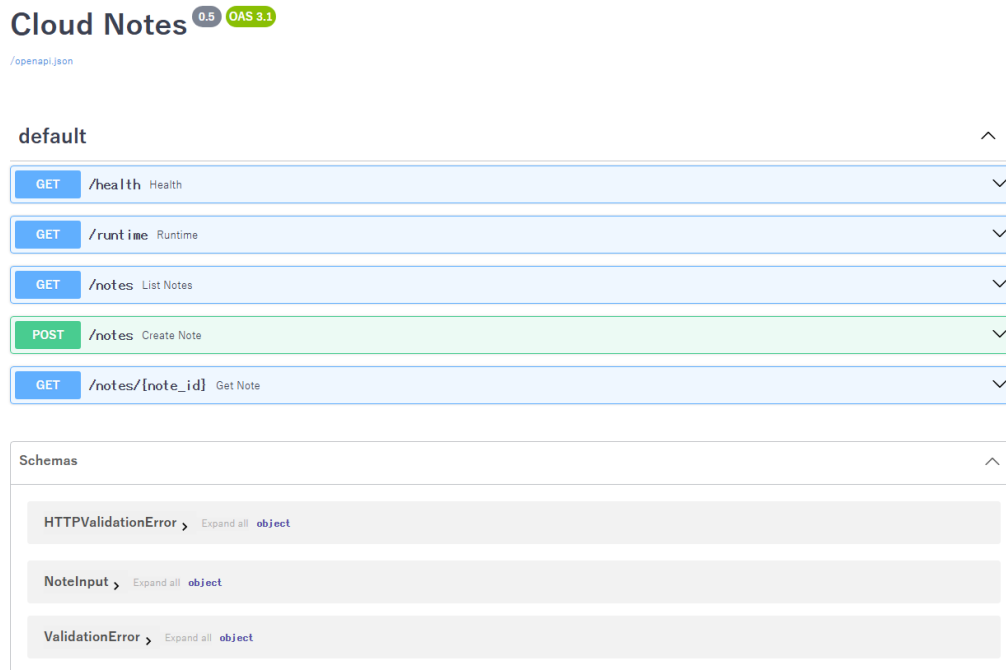
Perintah: `docker build`, `docker ps`, `docker exec cloudlab-api id`, `docker image inspect`. **Fungsi:** membuktikan image ada dan proses tidak root. **Cara kerja:** Docker membangun lapisan lalu menjalankan Uvicorn dalam container. **Baca:** tag image, UID 10001, pemetaan 8000:8000.



Docker Desktop menampilkan API Lab 05 aktif

Perintah: `docker run --name cloudlab-api-visual -d -p 127.0.0.1:8005:8000 -e APP_ENV=classroom cloud-notes-api:lab05`, lalu buka Docker Desktop > Containers dan jalankan `docker ps`. **Fungsi:** menunjukkan proses container dan port mapping dalam GUI. **Cara kerja:** host port 8005 dipakai khusus screenshot karena Lab 06

sedang memakai host port 8000; container tetap mendengar di 8000. **Baca:** titik hijau berarti proses berjalan, **8005:8000** berarti host→container; HTTP **/health** pada 8005 teruji 200. Pada praktik utama gunakan mapping/URL 8000 seperti command demo sebelumnya.



Swagger UI pada uji awal

Perintah: buka <http://127.0.0.1:8000/docs>. **Fungsi:** membaca kontrak FastAPI. **Cara kerja:** API menghasilkan OpenAPI saat dijalankan. **Baca:** /health, /runtime, /notes beserta metode.

```

Lab 05: respons API lokal

Invoke-RequestMethod /health, /runtime, POST /notes, GET /notes

GET /health
{"status":"ok","service":"cloud-notes-api"}

GET /runtime
{"environment":"classroom","storage":"memory"}

POST /notes
{"id":1,"title":"Docker","content":"Catatan kelas"}

GET /notes
{"value":[{"id":1,"title":"Docker","content":"Catatan kelas"}],"Count":1}

```

Respons API uji awal

Perintah: GET health/runtime, POST/GET notes. **Fungsi:** membuktikan aliran HTTP ke proses dalam container. **Cara kerja:** port host diteruskan ke Uvicorn. **Baca:** status sehat, `storage=memory`, ID baru, daftar catatan.

Cloud Services · hasil perintah lokal

Validasi dan reset data memori

Output aktual dari sesi uji; ditata ulang agar mudah dibaca pada modul.

```
POST /notes; input invalid; docker rm/run; GET /notes
```

```
POST /notes sebelum container diganti
```

```
{"id":1,"title":"Sementara","content":"Hilang saat container diganti"}
```

```
POST /notes dengan title kosong
```

```
HTTP 422
```

```
GET /notes setelah docker rm + docker run
```

```
[]
```

Validasi dan reset container awal

Perintah: POST judul kosong, `docker rm -f`, `docker run`, GET notes. **Fungsi:** membedakan validasi payload dari ketahanan data. **Cara kerja:** input tidak valid ditolak, sedangkan proses baru mulai dengan dictionary kosong. **Baca:** 422 dan `[]`.

Lab 05 - Docker API aktual

```
docker build cloud-notes-api:lab05 -> sukses
```

```
docker run -p 127.0.0.1:8000:8000 -> clouddlab-api aktif
```

```
GET /health -> HTTP 200 {'status': 'ok', 'service': 'cloud-notes-api'}
```

```
GET /runtime -> HTTP 200 {'environment': 'classroom', 'storage': 'memory'}
```

```
POST /notes -> HTTP 201 id=1 title=Docker
```

```
GET /notes -> HTTP 200 count=1
```

```
POST /notes title=" " -> HTTP 422
```

```
GET /notes/999999 -> HTTP 404
```

Output HTTP aktual ditata ulang agar terbaca. Password/token tidak direkam.

Uji ulang Docker/API Lab 05

Perintah: build/run dan GET/POST/GET dari daftar di atas. **Fungsi:** bukti run terbaru, termasuk judul spasi dan 404. **Cara kerja:** transkrip hasil Docker/HTTP aktual ditata ulang agar terbaca. **Baca:** 200, 201, 422, 404.

Cloud Notes 0.5 OAS 3.1

/openapi.json

default

GET	/health	Health	▼
GET	/runtime	Runtime	▼
GET	/notes	List Notes	▼
POST	/notes	Create Note	▼
GET	/notes/{note_id}	Get Note	▼

Schemas

HTTPValidationError > Expand all object

NoteInput > Expand all object

ValidationError > Expand all object

OpenAPI live dari container terbaru

Perintah: buka `/docs` saat container berjalan. **Fungsi:** memverifikasi browser dan port. **Cara kerja:** Chrome lokal memuat Swagger UI dari API aktual. **Baca:** versi 0.5 dan route yang tersedia.

```
Lab 05 - solusi /stats, image dibangun dari salinan kunci
GET /stats sebelum POST -> HTTP 200, 0 catatan, 0 karakter
POST /notes (Ops) -> HTTP 201
POST /notes (Incident 42) -> HTTP 201
GET /stats -> HTTP 200, 2 catatan, judul terpanjang 11
POST judul berisi spasi saja -> HTTP 422

Hasil Docker/API aktual ditata ulang agar terbaca; kode starter tetap tanpa endpoint tantangan.
```

Kunci stats yang sudah diuji

Perintah: build tag solusi sementara lalu GET `/stats` sebelum/sesudah dua POST. **Fungsi:** membuktikan kunci, tanpa memberi endpoint siap pakai di starter. **Cara kerja:** kode baru membaca `_notes` di bawah lock. **Baca:** 0/0 lalu 2/11, 422 untuk input spasi.

Kunci analisis, troubleshooting, dan Git

1. COPY/RUN bekerja saat build; CMD saat run. ENTRYPOINT dapat menetapkan executable utama, tetapi Dockerfile ini cukup dengan CMD.
2. `.dockerignore` mencegah `.env` masuk build context/image; `.gitignore` mencegahnya ikut commit.
3. `127.0.0.1:8000:8000` berarti host loopback port 8000 ke container port 8000; port kiri hanya dapat diakses lokal.
4. `Field(min_length=1)` menolak kosong, validator menolak whitespace saja, FastAPI mengembalikan 422.
5. Data `_notes` berada di memori proses; penggantian container menghilangkannya. Lab 06 memakai PostgreSQL/volume.

Gejala	Tindakan
Port 8000 terpakai	<code>docker ps</code> ; hentikan Lab 05 lama atau Lab 06 sesuai pemiliknya, jangan paksa hapus container lain.
404 pada <code>/stats</code>	Build ulang, jalankan tag <code>stats</code> , cek image container.
<code>/docs</code> belum siap	<code>docker logs cloudlab-api</code> , tunggu Uvicorn, cek port mapping.
POST spasi menghasilkan 201	Periksa bahwa image terbaru memuat validator <code>reject_blank_text</code> .
Data tidak hilang	Pastikan benar-benar <code>docker rm -f</code> lalu <code>docker run</code> , bukan hanya <code>docker restart</code> .

Simpan laporan di `hasil/lab05.md` dan bukti pribadi di `hasil/bukti/`. Dari root repo jalankan `git status --short`, `git diff --check`, `git add app Dockerfile .dockerignore requirements.txt hasil/lab05.md hasil/bukti`, `git diff --cached --name-only`, `git diff --cached --check`, baru commit/push. Cleanup `docker rm -f cloudlab-api` membebaskan port untuk Lab 06. **Teruji:** build image dari target, browser OpenAPI, health/runtime, POST/GET, whitespace 422, ID tak dikenal 404, container baru [], dan solusi stats 0/0→2/11 pada salinan sementara.